

birdclef-2026

8位解法

Rank	8
Team	kazumax
Author	kazumax
Topic ID	704770
Version	Japanese Translation

Source: <https://www.kaggle.com/competitions/birdclef-2026/discussion/704770>

Retrieved with Kaggle CLI on 2026-07-03.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

原文（Kaggle Discussion）：<https://www.kaggle.com/competitions/birdclef-2026/discussion/704770>

Public LB: 0.94751

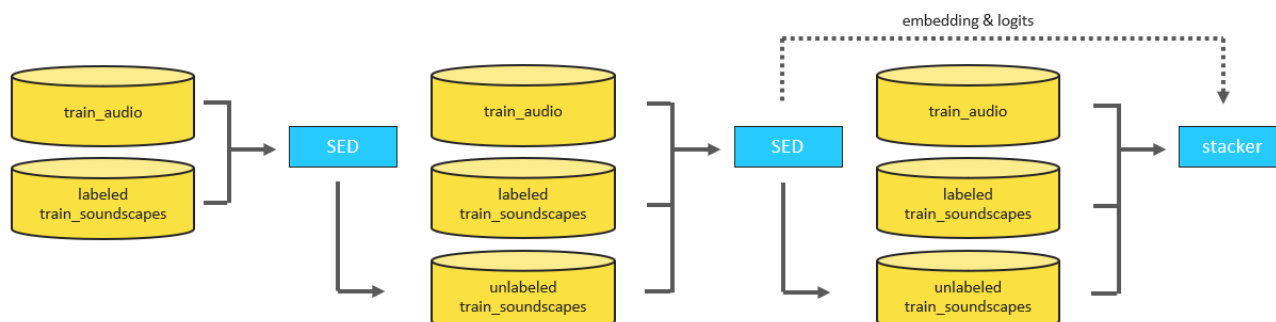
Private LB: 0.95675

まず、コンペティションの主催者の皆さん、そして期間中にアイデアを共有してくださったすべての方々に感謝します。

私の解法の出発点は、ここで共有された公開の蒸留SED（Sound Event Detection、音響イベント検出）のアイデアでした：

<https://www.kaggle.com/competitions/birdclef-2026/discussion/694479>

私はそのアイデアを、5秒間のPerch蒸留SEDモデルの出発点として利用し、その後、60秒間のサウンドスケープ構造、site/hour情報、時間的コンテキスト、およびクラスの事前分布（prior）をモデリングするためのスタッカーを追加しました。



TL;DR

私の最終サブミッションでは以下を使用しました：

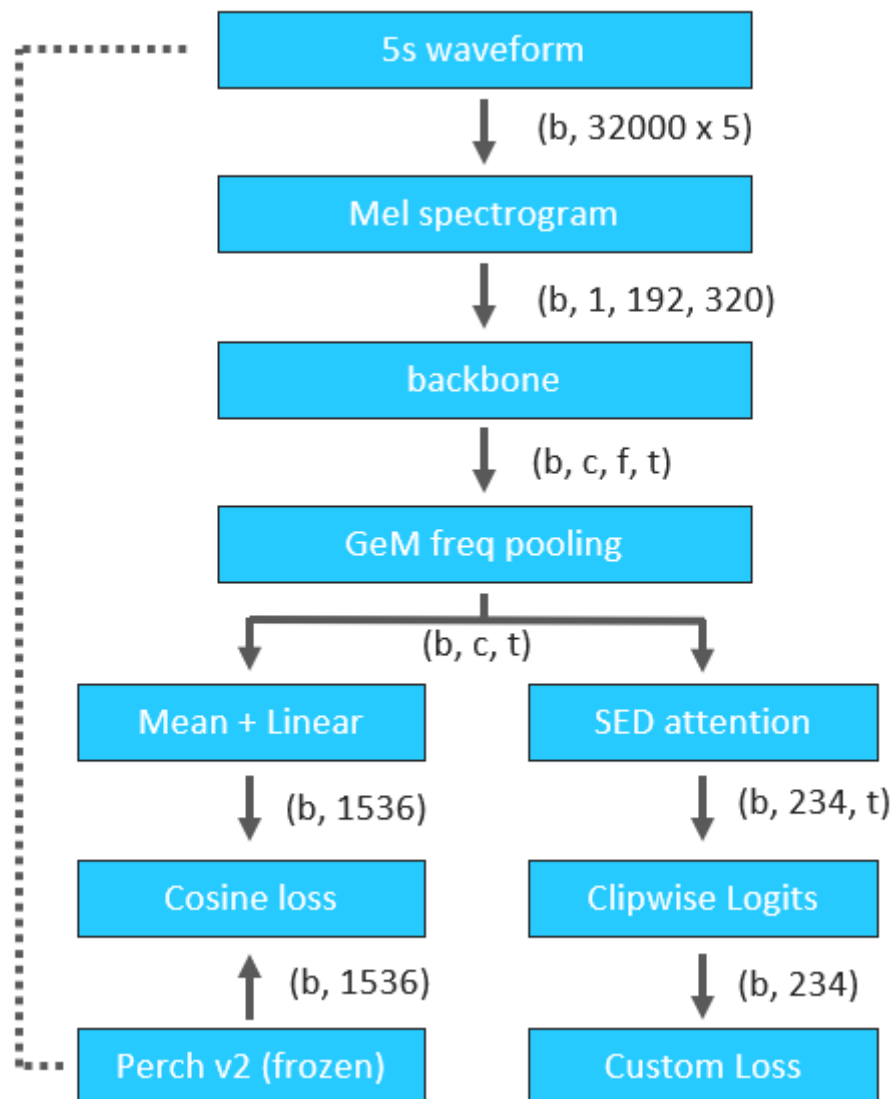
- 3つのtimmバックボーン（`efficientnet_v2_b3`、`regnety_016`、`maxxvitv2_nano_rw_256`）を用いた3-fold SEDモデル。
- 192 x 320 のdB-melスペクトログラムを持つ、5秒間・32 kHzの入力音声。
- frequency GeM poolingを備えたSEDヘッド。
- cosine lossを用いたオンラインのPerch v2埋め込み蒸留。
- 教師ありSED損失としてのSoft Cross Entropy + 弱いnegative-logitペナルティ。
- SED訓練用の擬似ラベル付きサウンドスケープ。
- 擬似ラベルを用いたNNスタッカーと、擬似ラベルを用いないツリースタッカー。
- sonotypeラベルとnon-sonotypeラベルで異なるスタッカーの分岐。
- SEDバックボーンにはOpenVINOによるCPU推論、ヘッドとスタッカーにはPyTorch。

Validation (検証)

私は3 foldを使用しました。ローカル検証では、ホールダウトした `train_audio` クリップと、ラベル付き `train_soundscapes` クリップに対して別々にmacro AUCを計算し、その2つのmacro AUC値の平均を主要な指標として使用しました。

このローカル指標は有用でしたがノイズが多かったため、最終的なモデル選定にはリーダーボードのフィードバックとアンサンブルの多様性も併用しました。

ベースSEDモデル



ベースモデルは、上述の公開蒸留SEDの議論に着想を得た5秒間のSEDモデルです。私の最終版はいくつかの点で異なります:

- 異なるバックボーン。
- MSEではなく cosine埋め込み蒸留。
- カスタムの教師あり損失。

- 教師ありのsegmentwise損失の重みを 0.0 に設定。
- 異なる波形長とmelスペクトログラムのサイズ。

各5秒クリップは 192 x 320 のdB-melスペクトログラムに変換され、timmバックボーンに通した後、frequency GeM poolingが適用されます:

```
backbone feature:      (B, C, F, T)
GeM frequency pooling: (B, C, T)
```

SEDのattentionヘッドは以下を出力しました:

```
segmentwise logits: (B, 234, T)
clipwise logits:    (B, 234)
```

教師あり損失はclipwise logitsに適用されました。segmentwise出力も引き続き生成されましたが、segmentwiseの教師あり損失の重みは 0.0 でした。

5秒間の訓練クリップの作成

コンペティションの音声はすべて既に32 kHzだったため、各SEDサンプルは5秒間、すなわち 160000 サンプルに固定しました。

train_audio については、訓練中はランダムな5秒間のクロップを使用しました。検証では最初の5秒間のセグメントを使用しました。

すべての train_soundscape ファイルは60秒間だったので、重複しない12個の5秒間クリップに分割しました。

SED擬似ラベル

私は、以前のSEDモデルを用いてラベルなしのサウンドスケープに擬似ラベルを付与しました。5秒間の擬似ラベル行は、その最大クラス確率が少なくとも 0.40 である場合にのみ保持しました。保持された行の中では、確率が少なくとも 0.10 のものをsoftターゲットとして保持し、残りはターゲット 0 としました。

また、train_audio の secondary label (副次ラベル) も使用しました。

Augmentation (データ拡張)

以下のaugmentationを使用しました:

- 波形augmentation: random shiftとgain
- スペクトログラム変換: FilterAugmentとSpecAugment
- 波形ミキシング: max_mixup_waveform と sumix_waveform

SED損失

最終的な教師ありSED損失は以下の通りでした:

```
loss = Soft Cross Entropy + 0.03 * weak negative-logit penalty
```

Soft Cross Entropy項は、234クラスにわたって `log_softmax` を適用し、multi-hot / softターゲットをサポートします。BCEとは異なり、この項はクリップ内でクラス同士を競合させます。私はターゲットベクトルを合計が1になるように正規化しなかったため、複数のラベルを持つクリップは複数の正の項を寄与させることができました。

negative-logitペナルティは、ターゲットが0であるクラスに対する小さな偽陽性抑制器でした。それらのlogitのいずれかが `-5.0` (sigmoid確率で約 `0.0067` に相当) を超えたときに線形のペナルティを加えました。このペナルティを平均する前に、最大値の上位5%を除去し、少数の極端な偽陽性が支配的にならないようにしました。

いくつかの損失の組み合わせと重みを試した結果、これが私にとって最良でした。

Perch蒸留

SEDモデルは、Google Perch v2の埋め込みに一致するようにも訓練されました:

```
Perch teacher:      Perch v2
target embedding:    1536 dims
distillation loss:   cosine
distillation weight: 10.0
```

これはPerchの埋め込みのみを使用し、Perchのクラス確率出力は使用しませんでした。

スタッカー

ベースSEDは各5秒チャンクを独立に予測します。スタッカーは、60秒間のシーケンスコンテキスト、site/hour情報、時間的な近傍特徴量、およびクラスの事前分布を追加しました。

テスト時:

```
5-second rows:      L = 12
base SED logits:     (L, 234)
```

訓練中は、低信頼度の擬似ラベル行が除外されるため `L` は変動するものでした。NNスタッカーは行をサウンドスケープファイルごとにグループ化し、各バッチ内でシーケンスをパディングしました。

私は2種類のスタッカーファミリーを使用しました:

- NNスタッカー: GRU / LSTMのシーケンスモデル。
- ツリースタッカー: LightGBM / XGBoostのモデル。

また、2つの出力モードを使用しました:

- `residual`: ベースSED logitsに対する補正を予測する。
- `direct`: 最終的なlogitsを直接予測する。

クロスバリデーションでは、residualスタッカーはnon-sonotypeラベルでより良く機能し、一方でdirect NNスタッカーはsonotypeラベルでより良く機能しました。最終サブミッションではそれに応じて使い分けました。

Prior特徴量

この解法において「prior (事前分布)」とは、ラベル付き `train_soundscape` から得たラベル頻度の推定値を意味します。各クラスについて、以下から頻度を計算しました:

- global frequency (全体頻度)
- hour
- site
- site-hour

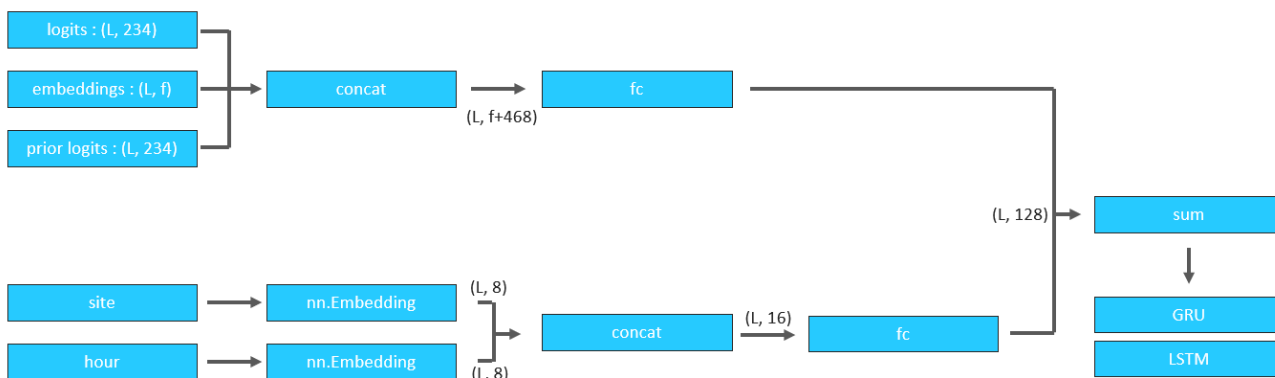
グループ頻度は、小さなsite/hourグループから極端なpriorが生じるのを避けるため、global frequencyに向けてスムージングしました。

私は2つの表現を使用しました:

```
prior probs = probability estimates
prior logits = logit(prior probs)
```

NNスタッカーはprior probabilitiesとprior logitsの両方を使用しました。ツリースタッカーはprior logitsを使用しました。これらの特徴量は、「このクラスはこのsite/hourで一般的である」あるいは「このクラスはここではほとんど観測されない」といった校正された期待値をスタッカーに与えました。

NNスタッカー



各5秒行について、NNスタッカーの入力は以下の通りでした:

```
base logits      (234)
SED embedding    (f)
prior logits     (234)
prior probs      (234)
```

ここで `f` はプールされたSED埋め込みの次元であり、バックボーンに依存します。

これらの特徴量は連結され、隠れサイズ128に射影されました。siteとhourは別々に埋め込まれ、連結され、隠れサイズ128に射影されて、音響特徴量の射影に加算されました:

```
feature projection: (L, input_dim) -> (L, 128)
site/hour metadata: (L, 16) -> (L, 128)
sum -> bidirectional GRU or LSTM -> output head
```

最終的なnon-sonotype NNスタッカーはGRUおよびLSTMのresidualモデルを使用しました。最終的なsonotype NNスタッカーはGRUおよびLSTMのdirectモデルを使用しました。

SEDの擬似ラベルとは異なり、NNスタッカーは各行につき上位5個の正の擬似クラスのみを保持しました:

- 擬似ラベル行は、その最大クラス確率が少なくとも 0.40 である場合にのみ保持する
- 確率が少なくとも 0.10 のクラスの中から、上位5個のみをsoftな正のターゲットとして保持する
- 保持された行のすべてのクラスについて損失を計算する
- 0.10 未満のクラス、および上位5個に選ばなかったクラスはターゲット 0 になる

ツリースタッカー

LightGBMとXGBoostについては、234個の別々のGBMを訓練するのはコストが高すぎたでしょう。そこで、class_id を特徴量として追加し、単一のGBMがすべてのクラスの補正を学習できるようにしました。

最終的なツリースタッカーはresidualモデルであり、non-sonotypeラベルにのみ適用しました。これらはSED埋め込みを使用せず、ベースSED logitsとそれらのlogitsから導出された特徴量に依存しました。

主なツリー特徴量は以下の通りでした:

- class idとクラス頻度統計
- 現在のbase logitとそれから導出された確率
- prior logit特徴量
- site/hour/end-secondのメタデータ
- 同一クラスの前後の5秒logits
- ベース予測からの行レベルのサマリ

この訓練テーブルは非常に不均衡だったため、負例をサンプリングしました。高スコアの誤ったクラスからのhard negativeとrandom negativeを使用しました。最終的なLightGBMはfoldあたり最大 500000 の訓練ペアを使用し、最終的なXGBoostは最大 1000000 を使用しました。

最終的なLGBM/XGBoostスタッカーには擬似ラベルを使用しませんでした。

最終スタッカーアンサンブル

non-sonotypeラベルの場合:

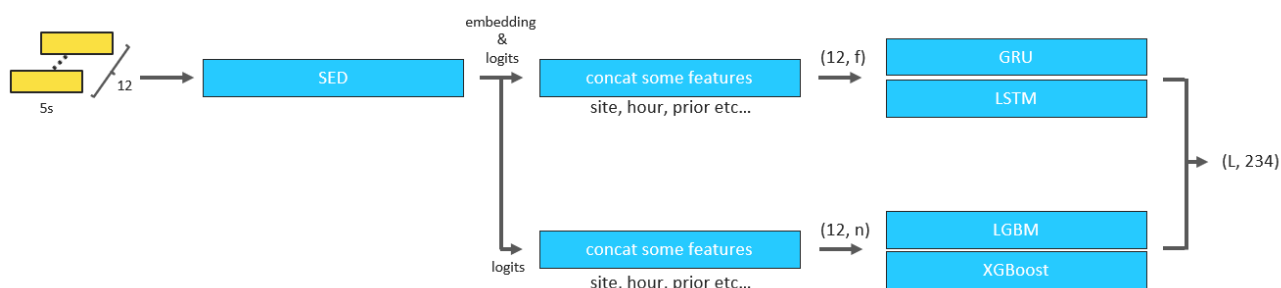
Branch	Type	Mode	Weight
GRU	NN	residual	0.5

Branch	Type	Mode	Weight
LSTM	NN	residual	1.0
LightGBM	tree	residual	0.5
XGBoost	tree	residual	1.0

sonotypeラベルの場合:

Branch	Type	Mode	Weight
GRU	NN	direct	0.5
LSTM	NN	direct	0.5

推論



推論は60秒間のサウンドスケープファイルに対して実行され、ファイルごとに12個のサブミッション行を生成しました。

私はSEDバックボーンをOpenVINO IRにエクスポートし、CPU上で実行しました。SEDヘッドとNNスタッカーはPyTorchのままにしました。

推論のフローは以下の通りでした:

1. 各60秒ファイルを5秒チャンクに分割する。
2. melスペクトログラムを構築する。
3. OpenVINO SEDバックボーンを実行する。
4. PyTorch SEDヘッドを実行する。
5. segmentwise予測を5秒間のサブミッション行に集約する。
6. base logits、embedding、prior、メタデータ、および時間的コンテキストからスタッカーの特徴量を構築する。
7. スタッカーアンサンブルを適用する。
8. 軽量な後処理を適用する。

後処理

私は2つの後処理ステップを使用しました。

第一に、重み [0.1, 0.2, 0.4, 0.2, 0.1] を用いた時間的スムージングを適用しました。各クラスについて、各5秒行を、現在の行を中心とした近傍行の加重平均で置き換えました。

第二に、sonotypeラベルについては、関連するsonotypeのバリエーションをグループ化し、各グループ内のすべてのラベルをそのグループ内の最大スコアに設定しました。

段階的なLBの向上

私は主要なコンポーネントの寄与も測定しました。以下のデルタは前の行に対する相対値です。

Stage	Public LB	Δ Public	Private LB	Δ Private
Baseline	0.90870	-	0.92496	-
+ Perch distillation	0.91565	+0.00695	0.94070	+0.01574
+ Pseudo-labeling	0.93608	+0.02043	0.94875	+0.00805
+ Stackers	0.94751	+0.01143	0.95675	+0.00800

私にとってうまくいかなかったこと

- 過去のBirdCLEFコンペティションのデータでSEDモデルをファインチューニングすること。
- SEDモデルに対する複数ラウンドの擬似ラベリング。
- 蒸留の際にPerchが予測したラベルを使用すること。

おわりに

私にとって、このコンペティションは単により強力な5秒間の音響モデルを作ることでは解けませんでした。ベースのSEDは重要でしたが、サウンドスケープのコンテキスト、site/hourのprior、時間的な連続性、そしてsonotype固有の挙動を第2段階でモデリングすることも同じくらい重要でした。

改めて、主催者の皆さん、そして議論、ノートブック、実験を共有して下さったすべての方々に感謝します。

余談ですが、実装のほとんどすべてはCodexで行いました。実験計画は自分で設計する必要がありましたが、それなしにはゴールドを獲得できなかったでしょう。この解法の執筆もCodexと一緒に行いました。ありがとう、Codex !

[inference_code](#)